



7

11

1320 sp

نظري

كلية الهندسة المعلوماتية

السنة الثالثة

مميزات لغوية إضافية في JavaScript

د. غاندي هسام



RB Informatics; 27/11/2025

لغات البرمجة

الأحداث (From even):

الأحداث التي يتم تشغيلها بواسطة النماذج تتيح لنا إجراء بعض المعالجة في الوقت المناسب استجابة لمدخلات المستخدم.

أكثر مستمعي جافا سكريبت شيوعاً للنماذج هو حدث الإرسال (submit).

الحدث	الوصف
فقدان التركيز	يتم تشغيله عندما يفقد عنصر من عناصر النموذج التركيز (أي ينتقل التحكم إلى عنصر مختلف أو ربما بسبب نقرة مفتاح Tab)
تغير (change)	تغيرت قيمة بعض حقول (<input> <textarea> <select>) قد يعني هذا أن المستخدم كتب شيئاً ما أو حدد خيار جديد.
التركيز (focus)	إضافةً إلى حدث فقدان التركيز يتم تشغيل هذا الحدث عندما يحصل عنصر ما على التركيز (ينقر المستخدم في الحقل أو ينتقل إليه باستخدام مفتاح Tab)
إعادة التعيين (reset)	يمكن إعادة تعيين نماذج HTML ثم تشغيل هذا الحدث عند حدوث ذلك
تحديد (select)	عندما يحدد المستخدم نصاً ما غالباً ما يستخدم هذا لمحاولة منع النسخ واللصق.
إرسال (submit)	عند إرسال النموذج يتم تشغيل هذا الحدث يمكننا إجراء بعض التحقق المسبق من النموذج في JavaScript قبل إرسال البيانات إلى الخادم

في القائمة 9.8 نستمتع إلى هذا الحدث على نموذج يحمل المعرف LoginForm. إذا كان حقل كلمة المرور الذي يحمل المعرف pw فارغاً فإننا نمنع إرسال البيانات إلى الخادم باستخدام `preventDefault()` وننبه المستخدم.

```
document.querySelector("#loginForm").addEventListener("submit",
function(e) {
    let pass = document.querySelector("#pw").value;
    if (pass=="") {
        alert ("enter a password");
        e.preventDefault();
    }
});
```

LISTING 9.8 Handling the submit event

النماذج في جافا سكريبت JavaScript:

لقد ذكر أنه ينبغي التحقق من صحة مدخلات نموذج المستخدم من جانب العميل ومن جانب الخادم.

❖ عند العمل مع النماذج في جافا سكريبت نهتم عادة بثلاثة أنواع من الأحداث:

- الحركة بين العناصر.
- تغيير البيانات داخل عنصر النموذج.
- تقديم النموذج النهائي.

استخدام JavaScript داخل النماذج هو أكثر من مجرد التحقق من صحة بيانات النموذج من جانب العميل كما يتم استخدام جافا سكريبت لتحسين تجربة المستخدم للنموذج النموذجي المستند إلى المتصفح.

الاستجابة لأحداث حركة الشكل:

```
// This Function is going to be called
// every the focus or blur events are
// triggered in one of our form's input
// elements.
Function setBackground (e){
    If(e.type == "focus") {
        e.target.style.backgroundColor = "#FFE393";
    }
    elseif (e.type == "blur"){
        e.target.style.backgroun.color="white";}}
```

ملاحظة: هنا نستخدم خاصية `style` بدلاً من خاصية `classList` بسبب وجود تعارضات في الخصوصية (`specificity`) بمعنى أن محددات الخصائص -Attribute Selectors- لها أولوية أعلى من محددات الفئات -classSelector

```
// setup the event listeners only after the DOM is loaded.
```

```
window.addEventListener("load",function){
```

```
    const cssSelector = "input[type = text],
```

```
        input[type = password]";
```

```
    const fields = document.querySelectorAll(cssSelector);
```

يحدد الحقول التي سيتم تغييرها.

```
    for(let f of fields){
```

```
        f.addEventListener("focus", setBackground);
```

```
        f.addEventListener("blur", setBackground);}}
```

ملاحظة: يخصص وظيفة (function) الـ setBackground() لتغيير لون خلفية عنصر التحكم بناءً على ما إذا كان العنصر في حالة التركيز (focus) أم لا.

التحقق من صحة الحقول الفارغة:

تتمثل طريقة التحقق من وجود حقل فارغ في جافا سكريبت في مقارنة قيمة ما بكل من القيمة الفارغة والسلسلة الفارغة ("").

```
const form = document.querySelector("#loginForm");
form.addEventListener("submit", (e) => {
    const fieldValue = document.querySelector("#username").value;
    if (fieldValue == null || fieldValue == "") {
        // the field was empty. Stop form submission
        e.preventDefault();
        // Now tell the user something went wrong
        console.log("you must enter a username");
    }
});
```

LISTING 9.10 A simple validation script to check for empty fields

تختلف عملية التحقق من صحة الحقول الفارغة قليلاً بالنسبة لمربعات الاختبار وأزرار الاختيار وقوائم التحديد.

```
const rememberMe = document.querySelector("#save");
if (!rememberMe.checked) {
    // if here then the checkbox was not checked
    console.log("Remember me was not checked");
}

const radios = document.querySelectorAll("input[name=region]");
for (let rad of radios) {
    if (rad.checked) {
        // if here then this radio button was selected
        console.log(rad.value + " was checked");
    }
}
```

التحقق من صحة الأرقام:

باستخدام `parseInt()` , `isNaN()` , `isFinite()` ويمكنك كتابة دالة التحقق من صحة الأرقام الخاصة بك.

```
Function is Numeric(n) {
  return !isNaN(parseFloat(n)) && isfinite(n); }
```

مميزات جافا سكريبت:

دوال المصفوفات:

لقد تعلمت بالفعل كيفية المرور على عناصر المصفوفة باستخدام حلقة For توفر دالة `forEach()` طريقة بديلة الكود

```
const paintings = [
  {title: "Girl with a Pearl Earring", artist: "Vermeer", year: 1665},
  {title: "Artist Holding a Thistle", artist: "Durer", year: 1493},
  {title: "Wheatfield with Crows", artist: "Van Gogh", year: 1890},
  {title: "Burial at Ornans", artist: "Courbet", year: 1849},
  {title: "Sunflowers", artist: "Van Gogh", year: 1889}
];
```

// version 2

```
for (let p of paintings) {
  console.log(p.title + ' by ' + p.artist);
}
```

// version 3a

```
paintings.forEach(function (p) {
  console.log(p.title + ' by ' + p.artist);
});
```

// version 3b - same as version 3a, but uses arrow syntax

```
paintings.forEach( (p) => {
  console.log(p.title + ' by ' + p.artist);
});
```

This function will be called for each element in the array

```
paintings.forEach( (p) => {
  console.log(p.title + ' by ' + p.artist);
});
```

Each element is passed in as an argument to the function.

```
const paintings = [
  {title: "Girl with a Pearl Earring", artist: "Vermeer"},
  {title: "Artist Holding a Thistle", artist: "Durer"},
  {title: "Wheatfield with Crows", artist: "Van Gogh"},
  {title: "Burial at Ornans", artist: "Courbet"},
  {title: "Sunflowers", artist: "Van Gogh"}
];
```

FIGURE 10.1 How `forEach()` works

- من أكثر سيناريوهات البرمجة شيوعاً مصفوفة من الكائنات هو إيجاد أول كائن تتطابق خاصيته مع شرط معين. وذلك يمكن تحديده باستخدام دالة `find()` الخاصة بكائن المصفوفة.

```
const courbet = Paintings.find(p => p.artist === "courbet");
```

```
console.log(courbet.title); // Burial at ornans.
```

- ماذا لو كنت مهتماً بالعثور يس نقط على أول تطابق بل على جمع التطابقات؟ في هذه الحالة يمكنك استخدام `.Filter()`

```
// vangoghs will be an array Containing two Painting objects
const vangoghs = Paintings. Filter( P => P.artist === 'Van Gogh');
```

- بطريقة مماثلة باستثناء أنها تنشئ مصفوفة جديدة بنفس الحجم ولكن تم تحويل قيمها بواسطة الدالة التي تم تمريرها `.map()`

This function will be called for each element in the array.

```
const options = paintings.map( p => `- ${p.title} (${p.artist})</li>` );

```

It will return a string containing a transformation of each array element ...

... which will generate this new array.

```
[
  "<li>Girl with a Pearl Earring (Vermeer)</li>",
  "<li>Artist Holding a Thistle (Durer)</li>",
  "<li>Wheatfield with Crows (Van Gogh)</li>",
  "<li>Burial at Ornans (Courbet)</li>",
  "<li>Sunflowers (Van Gogh)</li>"
];
```

- ترى استخداماً بديلاً لوظيفة `map()` بدلاً من إرجاع مصفوفة من السلاسل النصية بدلاً من ذلك إرجاع مصفوفة من عدد عناصر `Day`.

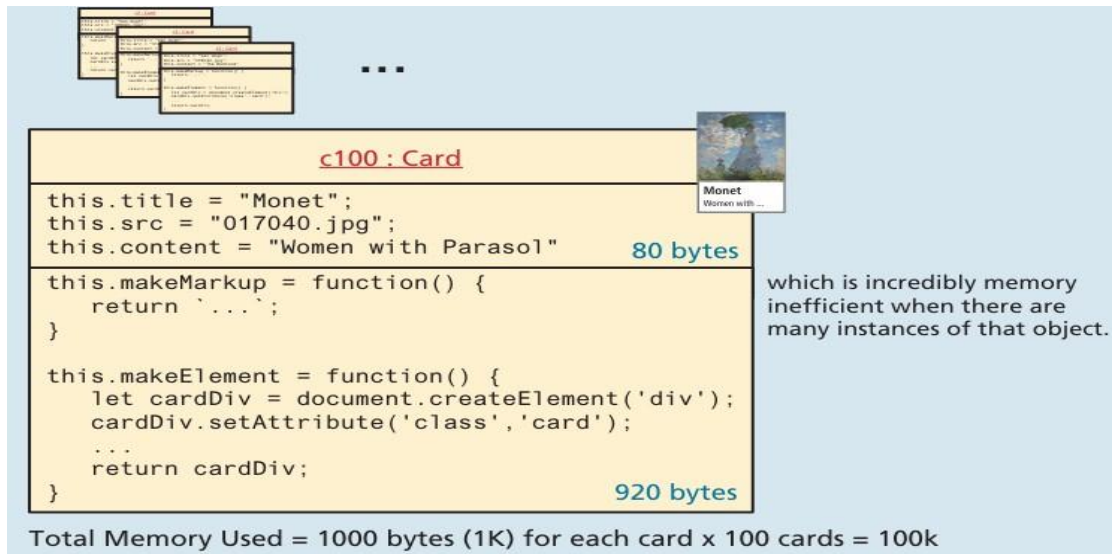
```
// create array of DOM nodes
const nodes = paintings.map( p => {
  let item = document.createElement("li");
  item.textContent = `${p.title} (${p.artist})`;
  return item;
});

// now add them to document
nodes.forEach( (n) => {
  document.querySelector("#parent").appendChild(n);
});
```

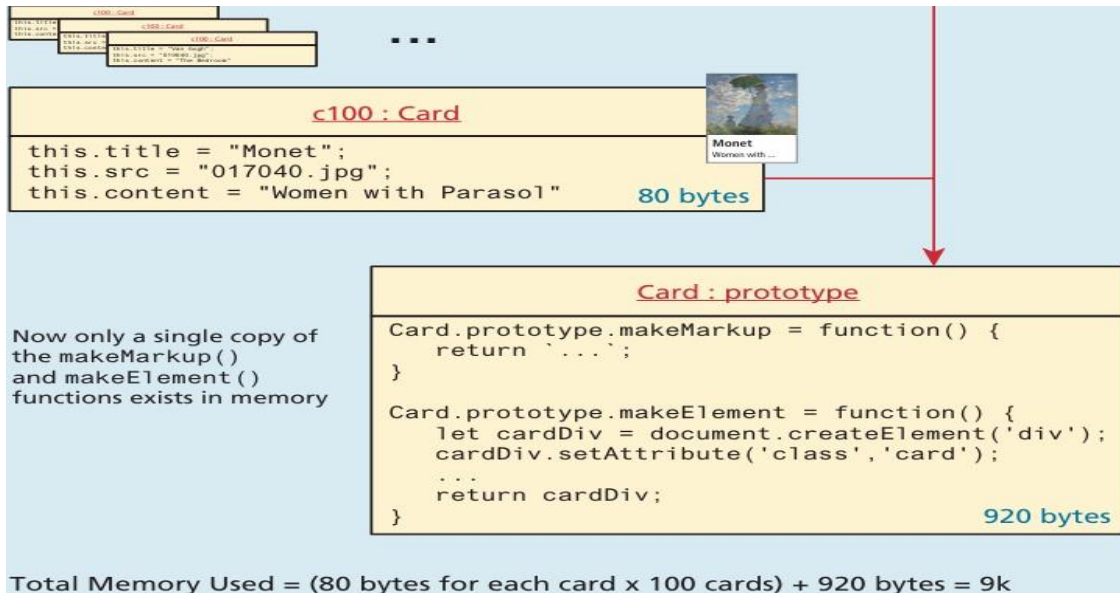
1. استخدام النماذج الأولية:

- تعد النماذج الأولية آلية بناء جملة أساسية في جافا سكريبت تستخدم لجعل جافا سكريبت تتصرف بشكل أقرب إلى لغة كائنية التوجه.
- ما يجعل خاصية النموذج الأولي قوية هو أن خصائص النموذج الأولي يتم تعريفها مرة واحدة لجمع مثيلات الكائن الذي تم إنشاؤه باستخدام الكلمة المفتاحية `new` من دالة البناء.
- يمكننا نقل تعريف طرق `makeElement()` و `make Markup()` من دالة البناء إلى النموذج الأولي.
- هذا النهج متفوق لكثير لأنه يحدد الطريقة مرة واحدة فقط بعض النظر عن عدد مثيلات البطاقة التي تم إنشاؤها.

بدون استخدام النموذج الأولي:



مع النموذج الأولي:



```
function Card(title, src, content) {
  this.title = title;
  this.src = src;
  this.content = content;
}
Card.prototype.makeMarkup = function() {
  return
    `<div class="card">
      
      <div>
        <h4>${this.title}</h4>
        <p>${this.content}</p>
      </div>
    </div>`;
};
```

```
Card.prototype.makeElement = function() {
  let cardDiv = document.createElement('div');
  cardDiv.setAttribute('class', 'card');
  let img = document.createElement('img');
  img.src = this.src;
  let div = document.createElement('div');
  let h4 = document.createElement('h4');
  h4.innerHTML = this.title;
  let p = document.createElement('p');
  p.innerHTML = this.content;

  div.appendChild(h4);
  div.appendChild(p);

  cardDiv.append(img);
  cardDiv.appendChild(div);
  return cardDiv;
};
```

```
// You use prototype functions as if they were declared in the object
const container = document.querySelector("#container");
const c1 = new Card("Van Gogh", "019170.jpg", "Self Portrait");
container.appendChild( c1.makeElement() );
```

استخدام النماذج الأولية لتوسيع نطاق الكائنات الأخرى:

```
String.prototype.countChars = function (c) {
  let count=0;
  for (let i=0;i<this.length;i++) {
    if (this.charAt(i) == c)
      count++;
  }
  return count;
}
```

LISTING 10.6 Extending a built-in object using the prototype

```
const msg = "HELLO WORLD";
console.log(msg + " has" + msg.countChars("L") + " letter L's");
```

2. الحصص الدراسية:

- توفر الفئة صيغة بديلة لمنشئ الدالة وامتدادها عبر النموذج الأولي الخاص بها.
- من المهم أن نتذكر أن فئة **JavaScript** الجديدة هذه ليست فئة مثل تلك الموجودة في **java**
- تستخدم إطار عمل **React** صيغة فئة **JavaScript**

```
const Card = class {
  constructor(title, src, content) {
    ...
  }
  makeMarkup() {
    ...
  }
  makeElement() {
    ...
  }
};
```

```
class Card {
  // constructor replaces the function constructor
  constructor(title, src, content) {
    this.title = title;
    this.src = src;
    this.content = content;
  }
  // class methods replace prototypes
  makeMarkup() {
    return
      `<div class="card">
        
        <div>
          <h4>${this.title}</h4>
          <P>${this.content}</P>
        </div>
      </div>`;
  }
}
```

Extending a class ↩

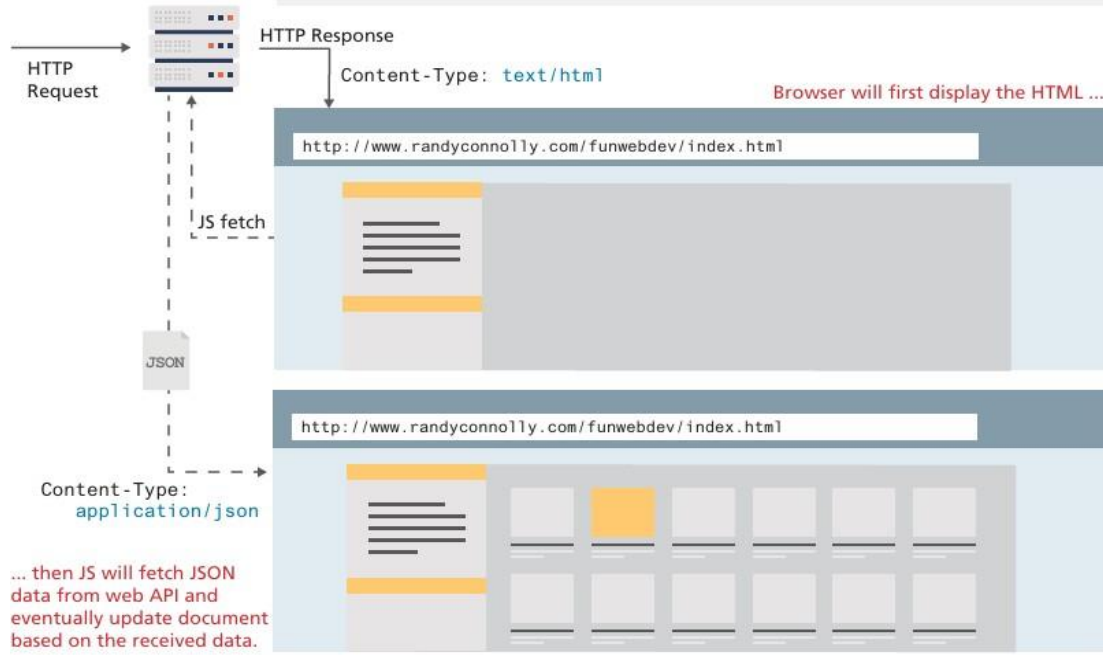
```
class AnimatedCard extends Card {
  constructor(title, src, content, effect) {
    super(title, src, content);
    this.effect=effect;
  }
  makeElement() {
    let element = super.makeElement();
    ...
  }
}

// notice that instance of AnimatedClass also has access to functions
// (e.g., makeMarkup) that are defined in the parent Card class
let x1 = new AnimatedCard("Monet", "017060.jpg", "Lilies", "fade");
console.log(x1.makeMarkup());
```

3. البرمجة غير المتزامنة باستخدام JavaScript:

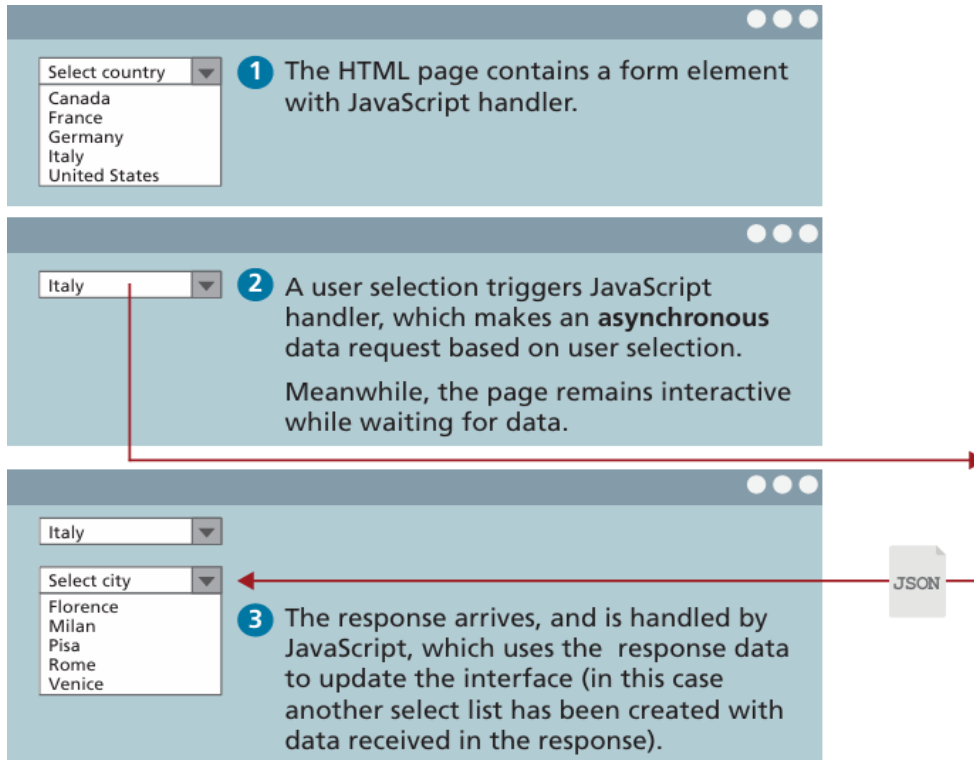
- في أنظمة التشغيل متعددة المهام يتم تحقيق التنفيذ غير المتزامن غالباً عبر الخيوط.
- هل لغة جافا سكريبت أحادية الخيوط بشكل أساسي؟ طريقة عملها هي أن المتصفح خيوط متعددة.
- يستخدم أحد هذه الخيوط لتنفيذ جافا سكريبت الخاصة بالصفحة أما الخيوط الأخرى تستخدم لأمر مثل المؤقت والعمل مع الملفات والوصول إلى كاميرا الويب أو جلب البيانات في الشبكة.
- يتفاعل كود جافا سكريبت الخاص بك مع هذه الخيوط من خلال وظائف الاتصال.
- تستخدم العديد من مواقع الويب المعاصرة طلبات بيانات جافا سكريبت غير المتزامنة لواجهات برمجة تطبيقات الويب.
- يسمح بتحديث الصفحة ديناميكياً دون الحاجة إلى طلبات فئة HTTP إضافية.
- واجهة برمجة تطبيقات الويب هي ببساطة مورد ويب يقوم بإرجاع البيانات بدلاً من HTML أو CSS أو JavaScript أو الصور

- باستخدام JavaScript **غير المتزامنة** من الممكن تحديث أقسام الصفحة عن طريق تقديم طلبات خاصة إلى الخادم في الخلفية.
- في الأصل كانت الطلبات غير المتزامنة في جافا سكريبت تتطلب برمجة معقدة باستخدام XMLHttpRequest الكائن الخاص بـ **ActiveX** أو **Mozilla** الخاص بـ **Internet Explorer**.
- المتصفحات الحديثة تدعم الآن وظيفة **fetch()** مما يعني أن المكتبات الخارجية لم تعد مطلوبة.



٤ واجهة برمجة التطبيقات:

لتوضيح عملية الجلب **fetch** ضع في اعتبارك سيناريو صفحة تحتوي على **<selects>** عنصر كما موضح بالشكل



- عندما يختار المستخدم من قائمة الدل تقوم الصفحة بإجراء طلب غير متزامن لاستمرار قائمة المدن الخاصة بتلك الدولة.

- يمكن بسهولة ترميز طلب الحصول على معلومات عن دولة إيطاليا كطلب URL على النحو التالي:

GET/api/cities.php?country=Italy

ملاحظة: نفترض هنا أن خدمة واجهة برمجة التطبيقات هذه مستضافة على خادمنا الخاص.

- لإجراء هذا الطلب باستخدام `fetch` يمكننا البدء بكتابة الكود التالي:

```
let cities = fetch('/api/cities.php?country=italy');
```

- إذن ما الذي تحتويه قائمة المدن بعد هذا الاستدعاء؟ قد تتوقع أن تحتوي على بيانات `JSON` المطلوبة لكنها لا تحتوي عليها
- لماذا لأن هذه الخدمة ستستغرق وقتاً لتنفيذ طلباً والاستجابة له.
- ما سيعيده الأمر `fetch` أعلاه بدلاً من ذلك هو كائن `promise` خاص.
- في الوقت الحالي اعتبر الوعد بمثابة بديل للبيانات التي ستصل في النهاية.
- عادةً ما يتم التعامل مع `promises` في جافا سكريبت عن طريق استدعاء طريقتين `then()` للاستجابة لوصول البيانات بنجاح و `catch()` للاستجابة لوصول غير ناجح.
- يتم تمرير دالة رد نداء إلى كل من هذه الطرق والتي يتم استدعاؤها عند حدوث أحداث النجاح والفشل.

```
console.log('before the fetch()');
let prom = fetch('/api/cities.php?country=italy');
prom.then( function(response) {
    // do something with this data
    console.warn('response received!!!');
});
console.log('after the then()');
```

- سيتم عرض رسالة (`responsereceived!`) في النهاية على الرغم من ظهورها في وقت سابق في الكود
- هذه سمة شائعة لطبيعة جافا سكريبت القائمة على الأحداث.

Notice that fetch returns a Promise object whose status is pending.

Once the data is received, the Promise status is changed to resolved.

It also includes other information such as HTTP headers and the actual data.

```
console.log('before the fetch()');
let prom = fetch('/api/cities.php?country=italy');
console.table(prom);
prom.then( function(response) {
    console.warn('response received!!!');
    console.log(prom);
});
console.log('after the then()');
```

- ومازلنا بحاجة إلى استخراج ملفات JSON من استجابة HTTP.
- يمكننا ذلك بجعل معالج then يعيد البيانات JSON قد ترغب في تجربة شيء مشابه لما يلي:

```
let data = prom.then( function(response) {
    return response.json();
});
console.log(data);
```

- لن يتم اخراج بيانات JSON بل سيتم إخراج كائن promise آخر معلق.
- يجب أن ينتظر الكود الخاص بك حتى يتلقى البيانات أي بعد استدعاء دالة الاستدعاء في معالج then() قبل أن يتم معالجتها يجب أن يبدو الكود الخاص بك على النحو التالي:

```
let data = cities.then( function(response) {
    return response.json();
});
data.then( function (data) {
    // now finally do something with the JSON data
    console.log(data);
});
```

This doesn't work ... why not?

```
let fetchedData;

fetch(url)

    .then( (resp) => resp.json() )
    .then( data => {
        fetchedData = data;
    });

displayData(fetchedData);
```

Execution order

1

2

4

5

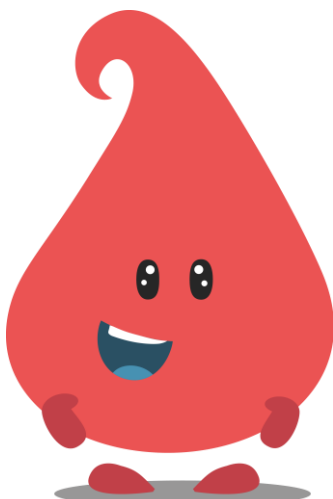
3

Remember that fetches are asynchronous ... the data will be received in the future.

fetchedData will be undefined when this line is executed.

Solution: move the call into the second then() handler.

FIGURE 10.14 Common fetch mistake



انتهت المحاضرة
<The end>